
DEPLOYMENT PLAN & PRODUCTION ARCHITECHTURE

UMHackathon 2026

umhackathon@um.edu.my

Technical Stack & Scalability

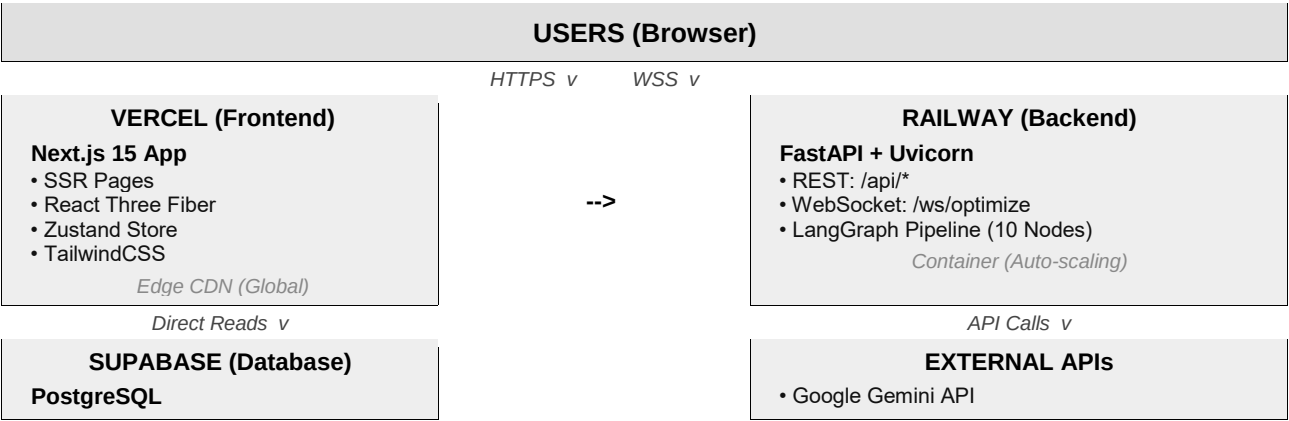
Technology Stack

The table below lists every layer of the system with its chosen technology, pinned version, and role in the overall architecture.

Layer	Technology	Version	Purpose
Frontend Framework	Next.js (React)	15.x	Server-side rendering, routing, and static optimisation
3D Visualisation	React Three Fiber + Drei	9.x / 10.x	Interactive 3D cube-based pipeline interface
State Management	Zustand (persist middleware)	4.x	Lightweight global state for session and stage data
Styling	TailwindCSS	4.x	Utility-first CSS framework for rapid UI development
Animation	Framer Motion	12.x	Smooth page transitions and micro-interactions
Backend Framework	FastAPI	Latest	High-performance async Python API with WebSocket support
Agentic Pipeline	LangGraph	Latest	Stateful, graph-based orchestration of 10 AI agent nodes
LLM Provider	Google Gemini (gemini-3.1-flash-lite)	3.1	All reasoning, analysis, optimisation, and validation tasks
Web Scraping	Firecrawl API	Latest	Structured web scraping for hotel data extraction
SEO Analysis	Google PageSpeed Insights API	v5	Lighthouse-based SEO, accessibility, and performance auditing
Database	Supabase (PostgreSQL)	Latest	Cloud-hosted relational database for optimisation records
Frontend Hosting	Vercel	—	Edge-deployed CDN with automatic SSL and preview deploys
Backend Hosting	Railway	—	Containerised Python deployment with auto-scaling

Production Architecture Diagram

The diagram below shows the end-to-end flow from browser to database, including the WebSocket channel used for real-time pipeline updates.



• Records • RLS Enabled Free Tier (500 MB)	• Firecrawl Scraping API • Google PageSpeed Insights API
---	--

Figure 1 — Production Architecture Overview

Scalability Considerations

Concern	Strategy
Frontend Load	Vercel deploys to a global Edge CDN. Static assets are cached at 300+ edge locations worldwide, ensuring sub-100 ms TTFB regardless of user location.
Backend Concurrency	FastAPI runs on Uvicorn (ASGI), supporting async I/O. Each WebSocket connection runs its own LangGraph pipeline concurrently without blocking other requests.
LLM Rate Limits	Google Gemini's free tier allows 15 RPM. For production scaling, the llm.py factory module is the single point of change — swap to a higher-quota key or different provider without touching any agent code.
Database Scaling	Supabase PostgreSQL supports connection pooling (PgBouncer). The optimization_records table uses JSONB columns for flexible schema evolution without migrations.
Pipeline Resilience	The LangGraph validator node implements a retry loop (max 2 retries) with feedback-driven self-correction, preventing infinite loops while maximising output quality.
Horizontal Scaling	Railway supports scaling to multiple container replicas. The backend is stateless (all persistent data is in Supabase), so any replica can handle any request.

CI/CD Pipeline & Automation

Pipeline Overview

Every push to main triggers GitHub Actions, which runs linting, type-checking, and all unit tests before auto-deploying to Vercel and Railway.



Figure 2 — CI/CD Pipeline Flow

Automated Testing

The project uses pytest for backend (Python 3.11) and Jest with the Next.js compiler for frontend, totalling 8 unit tests (6 backend + 2 frontend).

Test File	Tests	Components Covered
backend/tests/test_aeo_analyzer.py	3	analyze_image_alt_text(), analyze_semantics_and_specificity(), check_llms_txt()
backend/tests/test_agents.py	3	validator(), optimizer(), resimulator()
frontend/__tests__/store.test.ts	2	useSessionStore.setSession(), useSessionStore.addStage()

Execution Commands

```
# Backend tests (from backend/ directory)
python -m pytest tests -v
```

```
# Frontend tests (from frontend/ directory)
npm run test
```

GitHub Actions CI/CD Workflow

```
# .github/workflows/ci.yml
name: CI/CD Pipeline
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  # Backend Tests
  backend-tests:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: pip install -r requirements.txt && pip install pytest
      - name: Run unit tests
        run: python -m pytest tests -v

  # Frontend Tests
  frontend-tests:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - name: Install dependencies
        run: npm ci --legacy-peer-deps
      - name: Run unit tests
        run: npm run test
      - name: Build check
        run: npm run build

  # Auto Deploy
  deploy:
    needs: [backend-tests, frontend-tests]
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - name: Deploy triggers
        run: echo "Tests passed - Vercel and Railway auto-deploy from main."
```

Deployment Automation

Platform	Trigger	Process
Vercel (Frontend)	Push to main branch	Auto-builds Next.js app, deploys to global CDN, generates preview URLs for PRs
Railway (Backend)	Push to main branch	Auto-installs requirements.txt, runs Procfile start command, zero-downtime redeploy
GitHub Actions (CI)	Push or PR to main	Runs all 8 unit tests in parallel (backend + frontend jobs), blocks deploy on failure

Integration Security & Credential Management

Secret Management Strategy

All secrets and API keys are managed through environment variables and are never committed to the repository.

Control	Implementation
Git Exclusion	.gitignore contains .env* — all environment files are excluded from version control
Backend Secrets	Stored in Railway Dashboard > Variables tab. Injected at runtime into the container.
Frontend Secrets	Stored in Vercel Dashboard > Environment Variables. Only NEXT_PUBLIC_* prefixed vars are exposed to the browser (Next.js convention).
Database Credentials	Supabase URL and anon key stored as env vars. The anon key has Row Level Security (RLS) enforcement — it cannot bypass table policies even if leaked.
No Hardcoded Keys	All API keys (GOOGLE_API_KEY, FIRECRAWL_API_KEY, PAGESPEED_API_KEY) are loaded via python-dotenv using os.getenv().

Environment Variable Loading

Backend (llm.py)

```
from dotenv import load_dotenv
import os

load_dotenv() # Loads from .env in development, env vars in production

_API_KEY = os.getenv("GOOGLE_API_KEY")
```

Backend (supabase_client.py)

```
url: str = os.getenv("SUPABASE_URL")
key: str = os.getenv("SUPABASE_ANON_KEY")
```

Frontend (simulation.ts)

```
// NEXT_PUBLIC_ prefix ensures this is available client-side
let backendUrl = process.env.NEXT_PUBLIC_BACKEND_URL || "http://127.0.0.1:8000";
```

CORS & Network Security

The FastAPI backend restricts cross-origin access to only the known frontend domains:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000", # Local dev
        "http://127.0.0.1:3000", # Local dev
        os.getenv("FRONTEND_URL", ""), # Production Vercel domain
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Supabase Row Level Security (RLS)

```
ALTER TABLE optimization_records ENABLE ROW LEVEL SECURITY;

-- Only allow inserts (no anonymous deletes or updates)
CREATE POLICY "Allow anonymous inserts"
  ON optimization_records FOR INSERT WITH CHECK (true);

CREATE POLICY "Allow anonymous reads"
  ON optimization_records FOR SELECT USING (true);
```

Security Summary

Layer	Protection
Transport	All endpoints served over HTTPS/WSS (enforced by Vercel and Railway)
API Keys	Stored as platform env vars, loaded via <code>os.getenv()</code> , never in source code
Database	RLS policies restrict operations; anon key cannot bypass row-level rules
CORS	Backend only accepts requests from the whitelisted frontend domain
Git	<code>.env*</code> pattern in <code>.gitignore</code> prevents accidental credential commits
Frontend Exposure	Only <code>NEXT_PUBLIC_*</code> vars are exposed to browser; all sensitive keys remain server-side on Railway